CHECKPOINT 1 - PROJECT REPORT

C- Compiler: Scanner, Parser, Abstract Syntax Tree, and Error Recovery

Athina Manolakou, Ayesha Khan and Shifa Sheikh

1. WHAT HAS BEEN DONE

We built a C- compiler following the four incremental steps. First, we implemented a scanner using JFlex that recognises all required token types and reports lexical errors with line/column numbers. Second, we added a grammar-only parser using CUP and connected it to the scanner. Third, we embedded semantic actions to build an abstract syntax tree (AST) and added a -a option to output the tree to a .abs file. Finally, we incorporated error recovery so the parser can report multiple syntax errors in one run.

The compiler is invoked with ./CM -a file.cm (or without -a for parse-only). It was tested on the 5 sample programs (fac.cm, booltest.cm, gcd.cm, sort.cm, mutual.cm) and on 5 custom test files (1- 5.cm) that demonstrate parsing and error detection/recovery. Error recovery uses CUP's error symbol for declaration sequences, statement lists, parameter lists, and binary expressions. The parser skips erroneous constructs and continues, reporting each error and still generating a partial AST.

2. TECHNIQUES AND DESIGN

2.1 Scanner

The scanner specification in c.flex defines all token types required by the marking scheme: keywords (bool, else, if, int, return, void, while, true, false), operators (+, -, *, /, <, <=, >, >=, ==, !=, &&, ||, =, ~), punctuation (;, ,, (,), {, }), identifiers, integer literals, and block comments (/* ... */). Whitespace is skipped. Invalid characters are caught by a fallback rule and reported as lexical errors. Line and column numbers are tracked using JFlex's %line and

%column and passed into each token's Symbol object for accurate error reporting. The scanner is integrated with CUP by using the %cup directive and returning java_cup.runtime.Symbol tokens. Two-character operators are matched before single-character ones to avoid misclassification.

2.2 Parser

The parser grammar in c.cup follows the C-specification but simplifies expression handling by using CUP's precedence and associativity directives. For example:

```
precedence right ASSIGN;
precedence nonassoc LT, LE, GT, GE, EQ, NE;
precedence left PLUS, MINUS;
precedence left STAR, SLASH;
```

This correctly handles operator precedence and associativity while keeping the grammar concise. The grammar covers declarations (variables, arrays, functions, prototypes), parameters, statements (compound, if/else, while, return, expression statements, null statements), and expressions (binary, assignment, function calls, array indexing, identifiers, numbers).
Syntax errors are reported to stderr with line and column numbers (converted to 1-based) and a description of the unexpected token.

2.3 Abstract Syntax Tree

Each grammar rule contains a semantic action that instantiates the corresponding AST node from the absyn package. The root is a Program node holding a list of top-level declarations. Node types exist for all language constructs: declarations, types, statements, and expressions.

To display the tree, we implemented the Visitor pattern (AbsynVisitor) and a ShowTreeVisitor that prints the tree with indentation. When -a is used, Main.java retrieves the Program from the parser and writes the formatted tree to a .abs file with the same base name as the input.

2.4 Error Recovery

We enhanced CUP's default error reporting to include token type, value, and location. Recovery is done by inserting productions that use the error token, allowing the parser to synchronise after common mistakes. The following recovery points were added:

1. **Declaration recovery**: A general error SEMI production and a targeted case type_specifier ID error to handle missing semicolons after simple declarations.
2. **Statement sequence recovery**: In statement_list, an invalid statement can be skipped so later statements in the same block are parsed.
3. **Parameter list recovery**: A production for malformed parameter lists (e.g., an extra comma) that discards the erroneous part.
4. **Expression recovery**: A rule for binary expressions with a missing right operand (e.g., x = 3 + ;) that treats the missing part as an error and continues.

Recovered declarations that return null are omitted from the AST list, ensuring only valid constructs appear in the tree. The parser reports multiple errors in one run, as shown by 5.cm.

3. LESSONS LEARNED

- **Tool integration**: The order of generating files matters – CUP must run first to produce sym.java so JFlex can reference the correct terminal names. This dependency was correctly captured in the Makefile.
- **List construction**: Recursive rules like arg_list ::= arg_list COMMA expression required a helper method to append elements in the correct order, avoiding reversed lists.
- **Precedence directives**: Using CUP's precedence declarations simplified the grammar and eliminated shift/reduce conflicts without extra non-terminals.

- **Error recovery placement**: The location of recovery rules is important; overly broad rules can introduce new conflicts. Testing showed panic-mode recovery may skip more than intended, so we chose synchronisation points (semicolons, commas) to retain meaningful ASTs.

4. ASSUMPTIONS AND LIMITATIONS

We assume the C- specification grammar is authoritative and that input files have the .cm extension. The scanner does not support line comments (//) yet. Boolean operators (&&, ||, ~) are recognised lexically but are not yet wired into the expression grammar. Error recovery is based on a limited set of synchronisation tokens; while it handles many common errors, heavily corrupted input may still cause early termination near EOF.

5. POSSIBLE IMPROVEMENTS

- Add support for line comments in the scanner.
- Integrate boolean operators into the expression grammar.
- Enhance error messages to suggest expected tokens (e.g., “missing semicolon”).
- Add more recovery points for contexts like parenthesised expressions/malformed blocks.
- Provide a debug mode to display the token stream during scanning.

6. CONTRIBUTIONS

Shifa Sheikh implemented the scanner (JFlex specification, line/column tracking), initial grammar-only parser, Makefile (incremental steps 1 and 2). **Ayesha Khan** implemented AST node classes, semantic actions in CUP, ShowTreeVisitor, -a option in Main.java, README, and this report (step 3). **Athina Manolakou** implemented error recovery productions, enhanced error reporting, creation of the five test files (1.cm–5.cm), final integration (step 4).